

## Лекція 2.2

### Тема: Операції у мові JavaScript. Структури, що управляють, та організація циклів.

#### План

1. Операції у мові JavaScript.
2. Операції привласнення.
3. Арифметичні операції.
4. Операції порівняння.
5. Операції над рядками.
6. Умовні операції.
7. Булеві операції.
8. Операція typeof.
9. Операції зі структурами.
  2. Структури що управляються у мові JavaScript.
  3. Організація циклів у мові JavaScript.
  4. Оператор with.
  5. Оператор switch.

#### Зміст лекції

### 1. Операції у мові JavaScript

Основний сенс написання сценаріїв пов'язаний з необхідністю організації введення, обчислень, відображення або управління даними. До сих пір увага акцентувалася на способах відображення даних за допомогою JavaScript. Для створення хороших програм необхідно навчитися також оцінювати і змінювати дані, з якими JavaScript-сценарії мають справу. Інструментальні засоби, що реалізують подібну роботу, носять назву операцій.

**Операції** - суть символи і ідентифікатори, які представляють способи зміни даних або обчислення комбінацій виразів. Мова JavaScript підтримує як бінарні, так і унарні операції. Бінарні операції працюють з двома операндами в вираженні, наприклад,  $9 + x$ , тоді як для унарних операцій досить тільки одного операнда, наприклад,  $x++$ .

Обидва наведених прикладу демонструють арифметичні операції, їх використання буде зрозуміло тим, хто знайомий з основами математики. Інші типи операцій JavaScript оперують з рядками і логічними значеннями.

### 2. Операції присвоювання

Розглянемо операцію, про яку вже згадували раніше, - операцію присвоювання. Її основна функція полягає в присвоєнні змінної деякого значення, розміщуючи таким чином це значення в пам'яті.

Наприклад, вираз  $x = 20$  забезпечує присвоювання змінної  $x$  значення 20. Коли JavaScript зустрічає операцію присвоювання ( $=$ ), на початку аналізується права частина з метою визначення значення. Після цього розглядається ліва частина - вона повинна представляти місце для зберігання значення. Якщо зліва знаходиться змінна, їй присвоюється значення. В даному випадку  $x$  отримує певне значення 20. Операція присвоювання завжди виконується справа наліво, так що вираз  $20 = x$  викличе помилку в JavaScript, оскільки буде зроблена спроба привласнити 20 нового значення. Така дія неможлива, оскільки 20 не є змінною, а є цілим числом, значення якого не може бути змінено.

JavaScript підтримує 11 інших операцій привласнення, які фактично є комбінаціями операції присвоювання і арифметичних або порозрядних операцій. Скорочені версії операцій показані нижче:

*Комбінації операції присвоювання і арифметичних операцій:*

$x += y$  скорочений запис для  $x = x + y$   
 $x -= y$  скорочений запис для  $x = x - y$   
 $x *= y$  скорочений запис для  $x = x * y$   
 $x /= y$  скорочений запис для  $x = x / y$   
 $x \% = y$  скорочений запис для  $x = x \% y$

*Комбінації операції присвоювання і порозрядних операцій:*

$x \ll = y$  скорочений запис для  $x = x \ll y$   
 $x \gg = y$  скорочений запис для  $x = x \gg y$   
 $x \ggg = y$  скорочений запис для  $x = x \ggg y$   
 $x \& = y$  скорочений запис для  $x = x \& y$   
 $x \sim = y$  скорочений запис для  $x = x \sim y$   
 $x | = y$  скорочений запис для  $x = x | y$

### 3. Арифметичні операції

При роботі з числами використовуються арифметичні операції. До основних операцій цієї групи відносяться знак "плюс" (+), що підсумовує два значення, знак "мінус" (-), віднімає одне значення з іншого, зірочка (\*), перемножуються два значення, а також коса риска (/), яка позначає поділ одного значення на інше.

Коли JavaScript зустрічає одну з перерахованих операцій, виконується аналіз правої і лівої частин висловлювання, що має на меті відшукати значення, з якими доведеться працювати. У прикладі  $7 + 9$  JavaScript сприймає операцію плюс і переглядає обидві частини виразу, в результаті чого знаходиться 7 і 9. Далі операція плюс забезпечує підсумовування двох знайдених значень і встановлює результат виразу рівним 16. Використовуючи арифметичні операції в комбінації з операцією привласнення, можна задавати нове значення змінної:

$x = 7 + 9$

Змінна  $x$  буде тепер дорівнює 16, і її можна використовувати знову, в тому числі привласнювати їй нові значення:

$x = x + 1$

Подивіться на два останні приклади. Змінної  $x$  було присвоєно якесь значення. Потім  $x$  присвоюється нове значення - поточний (в даний момент дорівнює 16) плюс 1.

Збільшення значення змінної на 1 і потім присвоювання їй же цього нового значення є досить стандартну операцію. Остання настільки часто використовується в комп'ютерних програмах, що деякі мови включають спеціальні скорочені операції, щоб спростити запис збільшення і зменшення значення змінної. JavaScript - як раз відноситься до таких мов. Для запису інкремента застосовується ++, а для декремента --:

$++l$  - те ж саме, що  $l = l + 1$ ,  $--l$  - те ж саме, що  $l = l - 1$

Існує можливість використання цих скорочених операцій в префіксній і постфіксній формах. Таким чином змінюється порядок повернення значення виразу і установки нового значення.

Лістинг 2.1 демонструє роботу операцій інкремента і декремента.

Лістинг 2.1. Приклад операцій інкремента і декремента.

```
<html>
  <head>
    <title>Операції інкременту і декремента</title>
  </head>

  <body>
    <script type="text/javascript">
      var i = 0;
      var result = 0;
      document.write ( "Якщо i = 0,");
      document.write ( "i повертає своє значення");
      document.write ( "після інкрементування:");
      // префіксна форма інкременту
      result = ++ i;
      document.write (result);
      // Скинути значення змінної
```

```

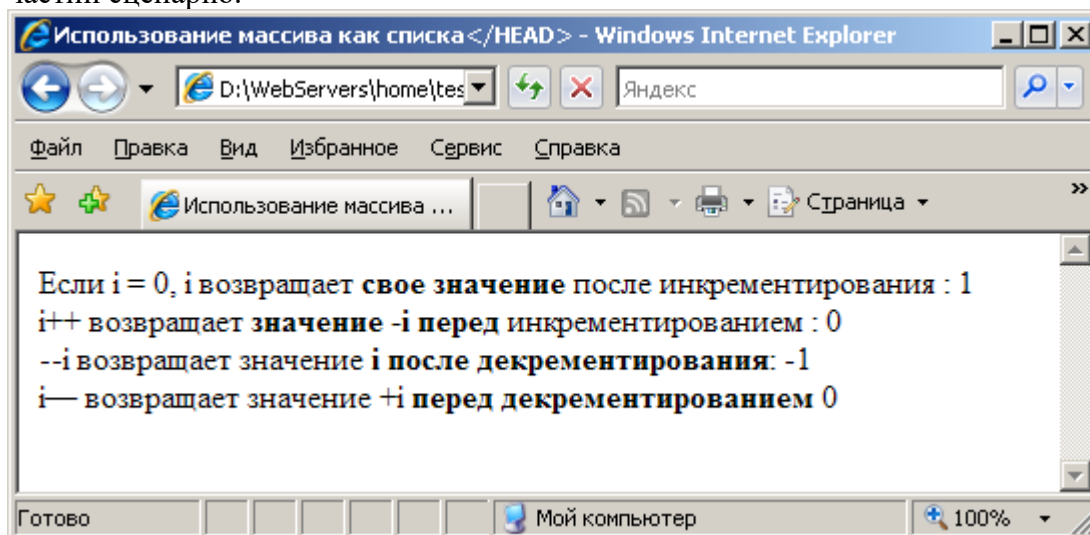
i = 0;
document.write ( "<br> i ++ повертає значення -i");
document.write ( "перед інкрементуванням:");
// Суфіксна форма інкременту
result = i ++;
document.write (result);
// Скинути значення змінної
i = 0;
document.write ( "<br> --i повертає значення i");
document.write ( "після декрементом:");
// префіксних форма декременту
result = --i;
document.write (result);
// Скинути значення змінної
i = 0;
document.write ( "<br> i-- повертає значення + i");
document.write ( "перед декрементом");
// Суфіксная форма декременту
result = i--;
document.write (result);
</script>
</body>
</html>

```

Після дослідження коду, представленого в лістингу 2.1. важливо зрозуміти, в який момент збільшується значення змінної *i* - перед або після обчислення виразу.

У першому прикладі змінна *result* отримує первинне значення *i* плюс 1. У другому прикладі результат встановлюється рівним початкового значення *i* до виконання інкременту *i*. Два наступних прикладу працюють аналогічним чином, однак стосовно операції декременту.

Результати обчислень цих чотирьох прикладів показані на рис. 2.2. Хоча це може здатися і непотрібним, розглянуті особливості принесуть безсумнівну користь при записі повторюваних частин сценарію.



МАЛЮНОК 2.2. Операції інкремента і декремента з лістингу 6.1.

Унарна операція заперечення (-) застосовується у випадках, коли потрібно замінити позитивне значення на негативне і навпаки. Вона названа унарною, тому що працює тільки з одним операндом. Якщо, наприклад, привласнити значення змінної *x* (*x* = 5), застосувати до *x*

операцію заперечення, а потім результат привласнити у ( $y = -x$ ), то значення у дорівнюватиме -5.

Протилежність також істинна, тобто при запереченні негативного числа результат буде позитивним. В обох прикладах операція заперечення не змінює значення змінної  $x$ , яке залишається рівним 5.

Операція взяття по модулю позначається знаком відсотка (%) Знайти модуль двох операндів означає знайти залишок після поділу першого операнда на другий. У прикладі  $x = 10\%$  3 змінна  $x$  отримає значення 1, оскільки результатом ділення 10 на 3 буде 3 із залишком 1. За допомогою операції взяття по модулю можна легко визначити, що одне число є множителем іншого: при цьому модуль цих двох чисел буде дорівнює 0. Це істинно, наприклад, для вираження  $x = 25\%$  5. 25 розділити на 5 дорівнює 5 без залишку, значить  $x$  дорівнюватиме 0.

#### 4. Операції порівняння

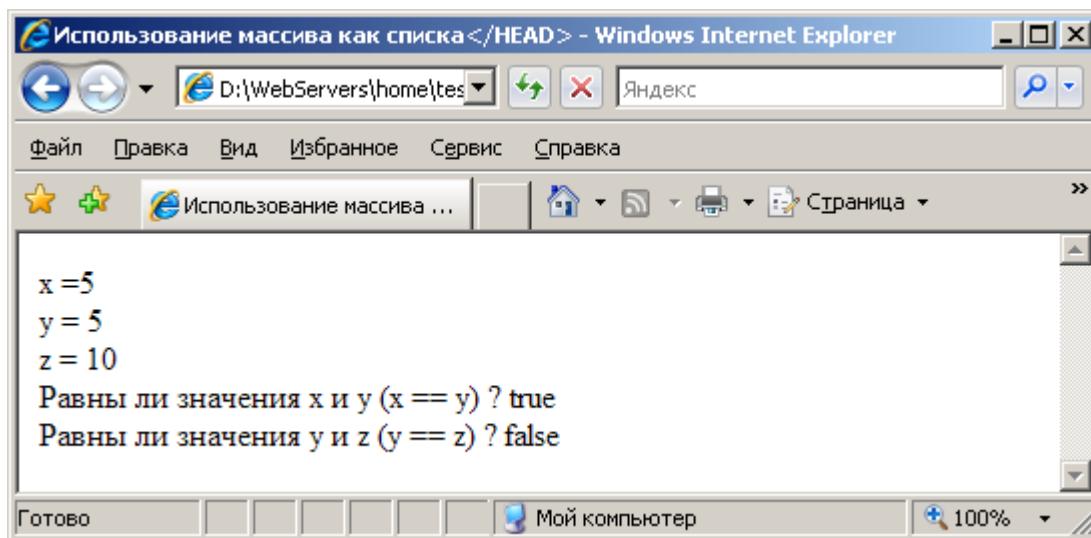
Операції порівняння використовуються саме для порівняння виразів. Вирази, які використовують операції порівняння, відповідають на питання, пов'язане з двома значеннями. Відповіддю може бути true або false.

Два знака "дорівнює" (==) позначають операцію рівності. Використовувати операцію рівності між двома операндами - означає визначити, чи є значення цих двох операндів рівними. Лістинг 2.2 демонструє спосіб відображення результатів при з'ясуванні рівності двох змінних.

#### Лістинг. Використання операції рівності.

```
<Html>
<head>
  <Title> Операції порівняння </title>
</head>
<body>
  <Script type="text/javascript">
    // Оголошення змінних
    var x = 5;
    var y = 5;
    var z = 10;
    // Відображення результатів
    document.write("x =" + x + "<br>");
    document.write("y =" + y + "<br>");
    document.write("z =" + z + "<br>");
    document.write("Чи рівні значення x і y (x == y)?");
    document.write(x == y);
    document.write("<br> Чи рівні значення y і z (y == z)?");
    document.writeln(y == z);
  </Script>
</Body>
</Html>
```

Необхідно переконатися, що при написанні програми використовується саме необхідна операція. Ще раз слід підкреслити, що операція рівності (==) перевіряє, чи є два значення рівними, тоді як операція присвоєння (=) встановлює значення змінної. У разі використання не тієї операції інтерпретатор JavaScript дасть про це знати. На мал. 2.3 показані результати виконання коду з лістингу 2.2.



МАЛЮНОК 2.3. Операція рівності з лістингу 2.2.

У табл. наведено список всіх операцій порівняння.

**Таблиця: Операції порівняння.**

| операція | опис                                                                                                                        |
|----------|-----------------------------------------------------------------------------------------------------------------------------|
| ==       | Операція рівності. Повертає true, якщо операнди рівні між собою.                                                            |
| !=       | Операція нерівності. Повертає true, якщо операнди не рівні між собою.                                                       |
| >        | Операція "більше". Повертає true, якщо значення лівого операнда більше значення правого операнда.                           |
| > =      | Операція "більше або дорівнює". Повертає true, якщо значення лівого операнда більше або дорівнює значення правого операнда. |
| <        | Операція "менше". Повертає true, якщо значення лівого операнда менше значення правого операнда.                             |
| <=       | Операція "менше або дорівнює". Повертає true, якщо значення лівого операнда менше або дорівнює значення правого операнда.   |

Операції порівняння зазвичай використовуються в JavaScript для прийняття рішень. Вони допомагають вибрати шлях, яким прямуватиме сценарій.

### 5. Строкові операції

Набір строкових операцій, доступний в JavaScript, включає всі операції порівняння і операцію конкатенації (+). За допомогою операції конкатенації рядка з'єднуються разом в одну довгу рядок (див. Лістинг 2.3).

#### Лістинг. Конкатенація рядків.

```
<html>
  <head>
    <title>JavaScript Unleashed</title>
  </head>

  <body>
    <script type="text/JavaScript">
      // Оголошення змінних
      var a = "www";
      var b = "company";
      var c = "com";
      var sumOfParts;
      var address1;
      var address2;
```

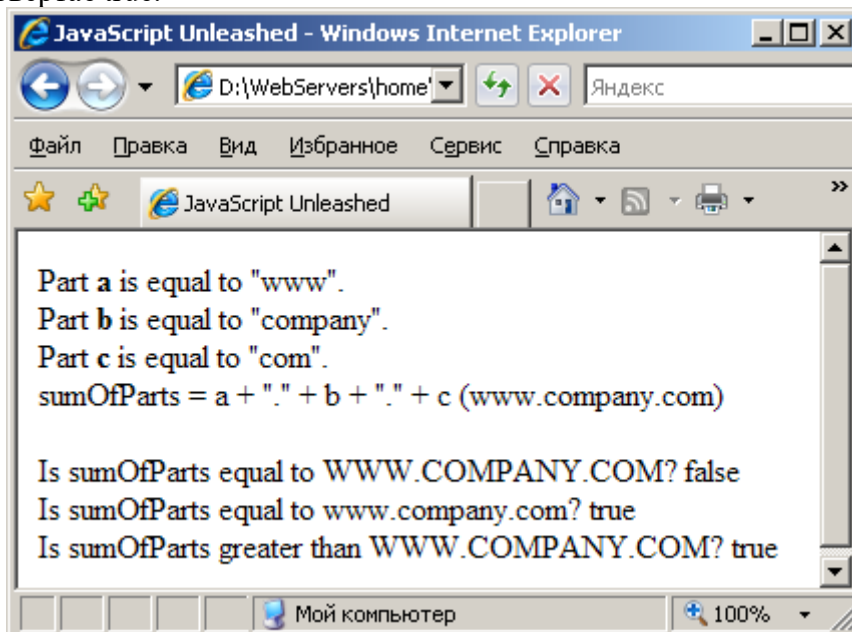
```

document.write ( "Part <b> a </b> is equal to \""+ a + "\".");
document.write ( "<br> Part <b> b </b> is equal to \""+ b + "\".");
document.write ( "<br> Part <b> з </b> is equal to \" " + c + "\".");
document.write ( "<br> sumOfParts = a + \". \" + b + \". \" + c = (www.company.com)");
sumOfParts = a + "." + b + "." + c;
address1 = "WWW.COMPANY.COM";
address2 = "www.company.com";
// Відображення результатів
document.write ( "<br> <br> Is sumOfParts equal to" + address1 + "?");
document.write (sumOfParts == address1);
document.write ( "<br> Is sumOfParts equal to" + address2 + "?");
document.write (sumOfParts == address2);
document.write ( "<br> Is sumOfParts greater than" + address1 + "?");
document.write (sumOfParts > address1);
</script>
</body>
</html>

```

Спочатку сценарій виконує ініціалізацію тріх змінних, що зберігають три частини Web-адреси. Потім всі три частини з'єднуються разом і поділяються необхідними точками, присутніми у всіх Web-адреси. Далі виконується перевірка, чи містить sumOfParts певну адресу Internet.

Малюнок 2.4 відображає той факт, що JavaScript при порівнянні рядків враховує регістр символів. Порівняння виконується зліва направо, по ASCII-кодами кожного символу в обох рядках. Якщо всі коди відповідають один одному, рядки рівні. Всі великі літери мають коди, менші, ніж їх еквіваленти в нижньому регістрі, тому і останнім порівняння в лістингу 2.3 повертає true.



Малюнок 2.3. Результат виконання коду з лістингу 2.3.

## 6. Умовні операції

Дві операції «?» та «:» використовуються для формування умовних виразів. Ці умовні операції виконують ті ж дії, що і оператор if. Умовний вираз повертає одне з двох значень, в залежності від логічного значення якогось вираження.

Наприклад, наступний умовний вираз буде повідомляти користувачеві; що він - мільйонний відвідувач сторінки:

```
var resultMsg = (numHits == 100000)? "Зі щитом": "На щиті";  
alert (resultMsg);
```

За умови, що numHits дорівнює 1000000 в будь-якому місці програми, цей вислів повертає рядок "Зі щитом!"; в іншому випадку - "На щиті". Другий рядок наведеного прикладу відображає результат для користувача за допомогою функції alert ()

Умовний вираз може застосовуватися для повернення даних будь-якого типу, наприклад, number або boolean. Наступне вираз повертає рядок або число в залежності від значення useString:

```
var result = useString? "Сім": 7;  
document.write (result);
```

## 7. Булеві операції

Булеві операції (також звані логічними) використовуються на додаток до виразів, що повертає логічні значення. Дія цих операцій простіше зрозуміти, вивчивши їх використання спільно з операціями порівняння. Таблиця 6.2 містить опис трьох булевих операцій. Наведені приклади з синтаксисом expression1 [operator] expression! і з [operator] expression.

**Таблиця: Булеві операції.**

| операції | опис                                                                                                                                                                                                                                                                                                                                                                                    |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| &&       | Логічна операція І (сполучення). Повертає true, якщо обидва вирази expression1 і expression2 мають значення true. В іншому випадку повертається false. Розглянемо приклади:<br>(1 > 0) && (2 > 1) повертає true.<br>(1 > 0) && (2 < 1) повертає false.                                                                                                                                  |
|          | Логічна операція АБО (диз'юнкція). Повертає true, якщо хоча б одне зі значень expression1 або expression2 одно true. Якщо жодне з expression1 і expression2 не дорівнює true, повертається false.<br>наприклад:<br>(1 > 0)    (2 < 1) повертає true.<br>(1 < 0)    (2 < 1) повертає false.                                                                                              |
| !        | Логічна операція НЕ (заперечення) унарная операція, яка повертає протилежне значення булева висловлювання. Якщо expression одно true, повертається false, а якщо expression - false, повертається true. Ця операція не змінює значення виразу, оскільки працює подібно арифметичної операції заперечення. Розглянемо приклади:<br>! (1 > 0) повертає false.<br>! (1 < 0) повертає true. |

## 8. Операція typeof

Операція typeof повертає тип даних, що зберігаються в операнде в поточний момент часу. Це виявляється особливо корисним при з'ясуванні, чи була визначена змінна. Розглянемо наступні приклади:

```
typeof unescape повертає рядок "function"  
typeof undefinedVariable возвращает рядок "undefined".  
typeof 33 повертає рядок "number",  
typeof "A String" повертає рядок "string",  
typeof true повертає рядок "boolean",  
typeof null повертає рядок "object".
```

## 9. Операції зі структурами даних

*Операції зі структурами даних* - термін, який використовується при класифікації двох операцій, необхідних для роботи зі структурами даних. Структури даних - це концептуальні засади, що застосовуються для зберігання однієї або більше базових елементів даних організованим способом. В JavaScript такими структурами є об'єкти, що групують дані для конкретних цілей.

Операція, вже знайома тим, хто мав справу з об'єктами, зазвичай називається "точка". Позначається вона точкою (.) і носить назву операцією взяття члена структури. Вона дозволяє звертатися до члена (яким може бути змінна, функція або цілий об'єкт), що належить певному об'єкту. Синтаксис операції такий:

```
objectName.variableName  
або  
objectName.functionName ()  
або  
objectName.anotherObject
```

Частина даних, до якої звертаються, повинна стояти праворуч від точки. Такий спосіб звернення до члена структури (до змінної, функції або об'єкту) зазвичай називається "точковою нотацією".

Операція індексування застосовується при зверненні до частини даних з масиву. Позначається вона парою квадратних дужок і дозволяє отримати доступ до будь-якого елемента масиву. Масиви в JavaScript реалізовані у вигляді об'єктів (див. Розділ 10). Нижче показаний синтаксис операції індексування:

```
arrayName [indexNumber]
```

Операція індексування передбачає використання цілого числа (індексу елемента масиву) `indexNumber`. `IndexNumber` визначає індекс в `arrayName` і забезпечує доступ до будь-якого члену масиву.

### Керуючі структури.

Проектування сценаріїв, які передбачають прийняття рішень під час виконання, - найцікавіша частина JavaScript. У сценарії, де потрібно приймати рішення на основі його поточного стану, просто забезпечують висновок питання і потім, в залежності від відповіді, вибирають шлях.

Розглянемо для прикладу планування ранку мого робочого дня. Мені доступні кілька різних варіантів, з яких на основі певних факторів я вибираю один. Я рідко готую їжу, тому найбільш важливе питання, яке я задаю собі щоранку, - чи не голодний я. Якщо так, я вибираю маршрут, що проходить через продуктовий магазин. До моменту завершення упаковки власного сніданку я, як правило, вже трохи спізнююся. Щоб заповнити втрачений час, я рухаюся у напрямку до швидкісного шосе, де можу збільшити швидкість. З іншого боку, якщо я не голодний (що трапляється досить рідко), то мину продуктового магазину і їду на роботу зі швидкістю на 10 миль в годину повільніше.

Використовуючи подібного роду планування, можна створювати і програми на JavaScript, що виконують різні операції в залежності від поточного стану. Наприклад, в розділі 6 розглядалося, як перевірити, чи є змінна дорівнює якомусь значенням чи навіть ряду значень. З використанням керуючих структур з'являється можливість змусити програму вибирати різні шляхи залежно від результату згаданої перевірки.

Оскільки розміри програм неухильно зростають, непогано було б врахувати важливий фактор - скільки часу буде потрібно клієнту для їх завантаження. Кращий спосіб заощадити на кількості вихідного тексту сценаріїв передбачає застосування операторів циклу. Вдавшись до допомоги циклів, можна досягти ситуації, коли всього лише кілька рядків програми забезпечують виконання великої кількості однотипних операцій. Це допоможе скоротити розмір сценаріїв і уникнути багаторазового набору одних і тих же команд. Крім іншого, в розділі демонструються способи підвищення ефективності сценаріїв.

умовні оператори



На початку теми розглядалася операція `?:`. Ця операція використовується для створення двокрокового процесу. Спочатку вираз оцінюється на предмет рівності `true` або `false`. Далі, в залежності від результату, повертається одне з двох значень. Якщо вираз `true`, повертається перше значення, а якщо - `false` - друге.

Зустрічаються сценарії, які приймають рішення в залежності від значення виразу, що використовує оператори `if` і `else`. Однак результат буде іншим. Замість повернення залежить від результату значення, програма вибирає один з двох шляхів виконання. Так, можна змусити JavaScript виконувати безліч разів особистих функцій, що залежать від будь-якої доступної інформації.

**if**

Оператор `if` відноситься до числа найбільш популярних. Кожна мова програмування містить його в тій чи іншій формі, і його використання уникнути, як правило, не вдається. Оператор `if` застосовується наступним чином:

```
if (умова) {  
    [Оператори]  
}
```

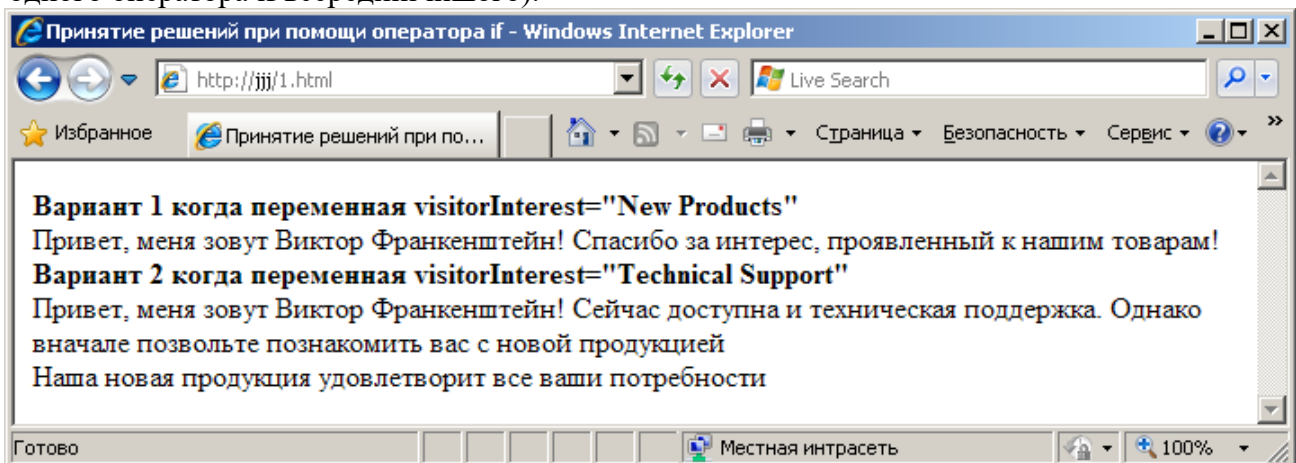
**В** як умову може вказуватися будь логічне вираз. Якщо результат умови дорівнює `true`, виконуються оператори, тому робота програми триває. Якщо умова повертає `false`, оператори ігноруються і робота триває.

У лістингу 2.4 наводиться сценарій, який відділ маркетингу міг би запропонувати розмістити на Web-сайті компанії. Сценарій починається з установки `visitorInterest` в одне з двох значень. В даному прикладі вибирається "Technical Support" (чи то пак, технічна підтримка) і коментуються альтернативні варіанти.

Сценарій доходить до першого оператора `if`, який перевіряє, чи дорівнює значення `visitorInterest` рядку "New Products" (з новий продукт). Останнє обчислене значення цього виразу - `false`, тому блок операторів після `if` ігнорується. Потім сценарій доходить до другого оператора `if` і порівнює значення `visitorInterest` з рядком "Technical Support". Вираз повертає `true`, що призводить до виконання коду, укладеного в фігурні дужки.

Незалежно від того, чому дорівнює значення `visitorInterest`, останній оператор завжди виконується, тому продавець Віктор Франкенштейн робить крок до відвідувача. Результуючий висновок показаний на рис. 2.4. Якщо `visitorInterest` привласнити якусь іншу значення, результат зміниться.

Зазвичай набір операторів полягає в фігурні дужки. Це надає сценарієм логічний вид і виявляється особливо корисним у випадку вкладених оператора `if` (тобто при використанні одного оператора `if` всередині іншого).



**МАЛЮНОК 2.4.** Віктор Фрапенштейн робить крок назустріч відвідувачеві.  
**Лістинг 2.4.** Використання оператора `if` для прийняття рішення.

```
<html>  
  <head>  
    <title>Прийняття рішень за допомогою оператора if</title>
```

```

</head>

<body>
  <script type="text/JavaScript">
    function f1 (visitorInterest) {
      // Оголошення змінних
      var visitorInterest;
      document.writeln ( "Привіт, мене звать Віктор Франкенштейн!");
      // Оцінка значення visitorInterest
      if (visitorInterest == "New Products") {
        document.writeln ( "Дякуємо за інтерес, проявлений до наших товарів!");
      }
      if (visitorInterest == "Technical Support") {
        document.writeln ( "Зараз доступна і технічна підтримка.");
        document.writeln ( "Однак спочатку дозвольте познайомити вас");
        document.writeln ( "з новою продукцією");
        document.writeln ( "<br> Наша нова продукція задовольнить всі ваші"); -
        document.writeln ( "потреби");
      }
    }
    document.write ( "<b> Варіант 1 коли змінна visitorInterest = \" New Product
s \"<br> </b>");
    f1 ( "New Products");
    document.write ( "<b> <br> Варіант 2 коли змінна visitorInterest = \" Techni
cal Support \"</b> <br>");
    f1 ( "Technical Support");
  </script>
</body>
</html>

```

Лістинг 2.5 демонструє використання логічних змінних для вибору шляху, яким підсценарій. Перший оператор if оцінює змінну needsInfo. В даному випадку needsInfo мала значення true, так що JavaScript вибирає перший блок if і продовжує роботу в ньому, тобто виводить на екран "Наші продукти використовує весь чир." Далі черга доходить до другого (вкладеного) блоку if, і оцінюється needsMoreInfo.

### Лістинг 2.5. Вкладені оператори if.

```

<html>
  <head>
    <title>Прийняття рішень за допомогою оператора if</title>
  </head>

  <body>
    <script type="text/JavaScript">
      // Оголошення змінних
      var needsInfo;
      var needsMoreInfo;
      // Якщо встановити одну з наведених нижче змінних в false, висновок змінитися.
      needsInfo = true;
      needsMoreInfo = true;
      document.writeln ( "Я працюю на кращих продуктах Interneshen1.");
    </script>
  </body>
</html>

```

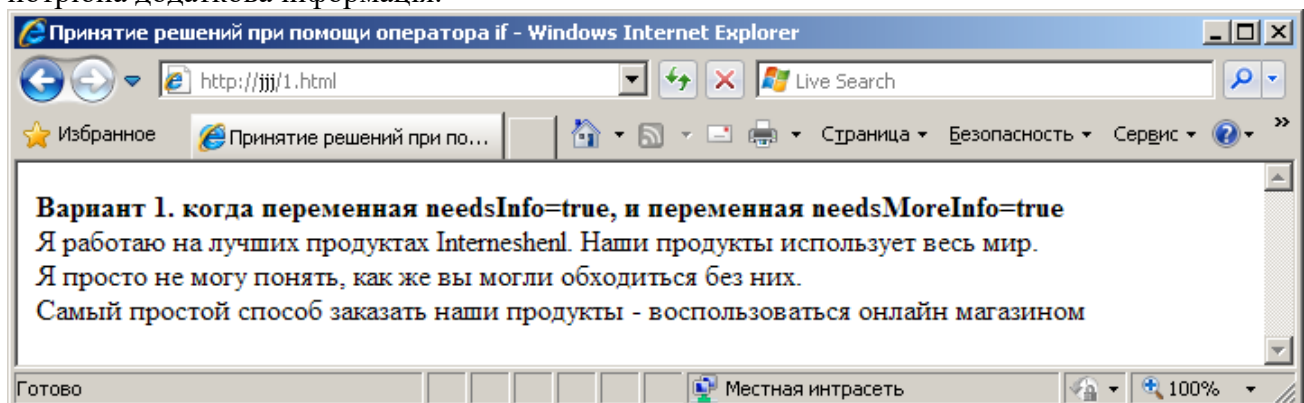
```

    if (needsInfo) {
        document.writeln ( "Наші продукти використовує весь світ.");
    if (needsMoreInfo) {
        document.writeln ( "<br> Я просто не можу зрозуміти, як же ви могли");
        document.writeln ( "обходитися без них.");}
        document.writeln ( "<br> Найпростіший спосіб замовити наші продукти - скорис-
татися");
        document.writeln ( "онлайн магазином");
    }
</script>
</body>
</html>

```

Знову-таки, повернене значення - true, тому оператори всередині другого блоку будуть виконуватися. Після завершення вкладеного блоку if JavaScript виконує всі залишилися інструкції першого блоку. Зверніть увагу, як фігурні дужки допомагають відрізняти окремі блоки програми один від одного.

З рис. 2.4 видно, що рядки програми виконувалися в порядку їх написання. Змінюючи значення needsInfo і needsMoreInfo, можна отримати три різних результату. Пам'ятайте, що якщо needsInfo привласнити значення false, програма не зможе перейти до другого блоку if. У цьому випадку значення needsMoreInfo ролі не грає, оскільки оцінюватися воно не буде. Вибачте, Віктор, але якщо відвідувачеві взагалі не потрібна інформація, йому вже точно не потрібна додаткова інформація.



МАЛЮНОК 2.4. Один з трьох можливих результатів.

if ..else

Іноді однієї конструкції if виявляється недостатньо. Часто потрібне також зарезервувати набір операторів, які будуть виконуватися в разі, коли умовний вираз повертає false. Це можна зробити, додавши ще один блок безпосередньо після блоку if:

```

if (умова) {
    оператори } else {
    оператори }

```

Крім того, є можливість скомбінувати частина else з іншим оператором if. Використання такого методу дозволяє оцінити кілька різних можливих сценаріїв перед виконанням потрібної операції. Витонченість даного методу полягає в тому, що в кінці можна визначити той же сегментом else. Формат згаданого типу оператора виглядає так:

```

if (умова) {
    оператори} else if (умова) {
    оператори } else {
    оператори }

```

Що може бути гірше прискіпливого комп'ютера? Лістинг 2.6 демонструє, як можна створити інтерактивну Web-сторінку за допомогою всього лише кількох рядків коду. Тут

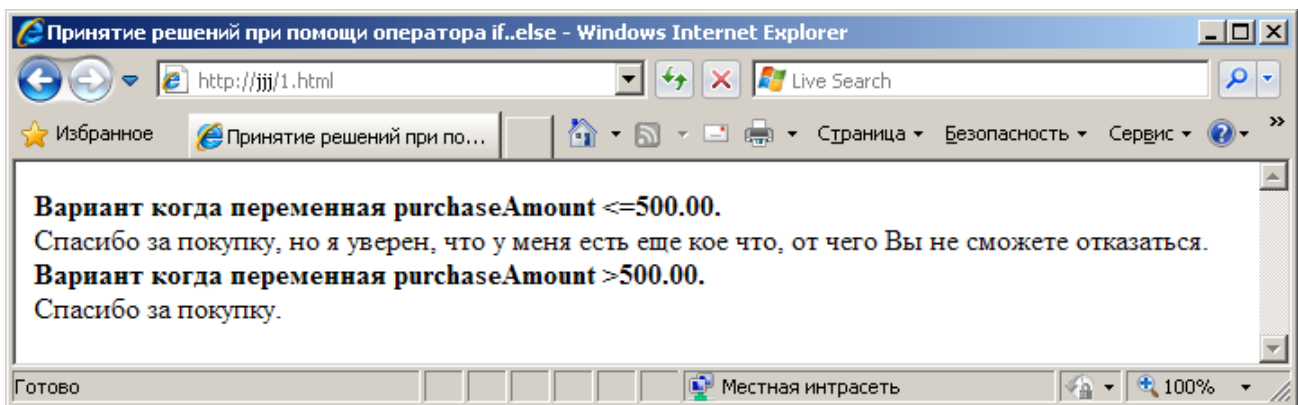
жорстко задано значення ключа purchaseAmount яке визначає хід сценарію. Для справжнього взаємодіювання сценарію з користувачами потрібно забезпечити можливість для користувача введення. В такому випадку purchaseAmount може приймати будь-які значення, в залежності від бажання замовника. Це робиться за допомогою JavaScript і стандартних вхідних HTML-об'єктів, таких як перемикачі та текстові об'єкти. У главі 17 ця проблема обговорюється більш детально.

У розглянутому прикладі клієнт не зволив витратити достатню суму, щоб задовольнити Віктора Франкешптейна. Умовний оператор отримує значення false, і програма переходить на блок else. Подивившись на рис. 7.3, можна на власні очі перекоонатися, як наполегливий продавець "розігріває" потенційного клієнта.

**Лістинг 2.6. Блок else, відповідний значенням false.**

```
<html>
  <head>
    <title>Прийняття рішень за допомогою оператора if..else</title>
  </head>

  <body>
    <script type="text/JavaScript">
      // Оголошення змінних
      var purchaseAmount;
      purchaseAmount = 10.00;
      document.writeln ( "<b> Варіант коли змінна purchaseAmount <= 500,00. <br> </
b>");
      if (purchaseAmount> 500.00) {
        document.writeln ( "Спасибі за покупку.");
      } Else {
        document.writeln ( "Спасибі за покупку, але я впевнений, що у мене є");
        document.writeln ( "ще дещо що, від чого Ви не зможете відмовитися.");
      }
      purchaseAmount = 510.00;
      document.writeln ( "<b> <br> Варіант коли змінна purchaseAmount> 500.00. <br>
</ b>");
      if (purchaseAmount> 500.00) {
        document.writeln ( "Спасибі за покупку.");
      } Else {
        document.writeln ( "Спасибі за покупку, але я впевнений, що у мене є");
        document.writeln ( "ще дещо що, від чого Ви не зможете відмовитися.");
      }
    </script>
  </body>
</html>
```



**МАЛЮНОК 2.5. Один з двох можливих результатів.**

### Оператори організації циклів.

Організація циклу всередині сценарію переслідує широкий спектр цілей. Розглянемо одне просте, але дуже поширене застосування циклу. Наприклад, написання програми, що відображає числа від 0 до 9, - на перший погляд, швидка і проста задача. Просто записуються 10 команд для відображення кожного числа:

```
document.writeln ( "0");  
document.writeln ( "1");  
document.writeln ( "9");
```

Це легко зробити для чисел від 0 до 9, але що якщо буде потрібно порахувати до 1000? Можна піти тим же способом, але тоді програма зажадає набагато більше часу на написання і завантаження в браузер. Найкращий спосіб рахунку до 1000 або будь-якого іншого числа полягає в використанні тієї ж самої команди відображення, але зі змінною замість літерала. В даному випадку потрібно забезпечити тільки одну річ - інкрементувати змінну і повторювати команду відображення. JavaScript має відповідними інструментами.

#### for

Оператор for використовується для організації циклу в сценарії. Перед детальним вивченням оператора for на основі лістингу 2.7, розглянемо його узагальнений синтаксис:

**for ([вираження\_ініціалізації]; [вираження\_умовія]; [вираження\_цікла]) {  
оператори }**

Три вираження, укладені в квадратні дужки, не є обов'язковими, проте, якщо навіть прибрати одне з них, крапка з комою все ж буде потрібно. Саме крапка з комою забезпечує належне місце для кожного виразу.

Вираз ініціалізації зазвичай застосовується з метою ініціалізації змінної і навіть оголошення, що конкретна змінна є лічильником циклу. Вираз умови повинно мати значення true перед кожним виконанням операторів, укладених у фігурні дужки. Нарешті, вираз циклу, як правило, інкрементує або декрементується значення змінної, яка використовується в якості лічильника циклу.

Лістинг 2.7 демонструє використання оператора for для організації рахунку від 0 до 99 з відображенням кожного числа. Для того щоб вмістити результат на стандартній сторінці, після кожних 10 чисел виводиться символ нового рядка.

#### Лістинг 2.7. Використання циклу for для рахунку від 0 до 99.

```
<html>  
  <body>  
    <script type="text/javascript">  
      // Вивести заголовну частину  
      document.write ( "Числа від 0 до 99:");  
      document.write ( '<hr size = " 0 "width ="50%" align ="left">');  
      for (var i = 0; i <100; ++i) {  
        //Символ нового рядка після кожного 10-го числа  
        if (i% 10 == 0) {
```

```

document.write ( '<br> ');}
document.write (i + ",");}
//останнє число, отже,
// завершується. . .
document.write ( "<br><br> Після завершення Циклу i одне:" + i)
</script>
</body>
</html>

```



#### МАЛЮНОК 2.6. Результат 100-кратного виконання циклу.

У лістингу 2.5 порядок виконання операторів виглядає так: вираз ініціалізації оголошує змінну *i* і встановлює її значення рівним 0. Потім змінна *i* перевіряється на предмет, чи менше вона 100. Коли *i* все ще дорівнює 0, вираз умови повертає true і програма виконує оператор; в фігурних дужках. Як тільки програма виконає всі оператори і досягне закриває фігурної дужки, вона обчислює вираз циклу ++ *i*. При цьому *i* збільшується на 1 і, таким чином, виконання першого повного циклу завершується.

Процес починається спочатку. Цього разу JavaScript знає, що виконувати розділ ініціалізації для циклу вже не потрібно. Вираз умови обчислюється знову. Якщо *i* все ще менше 100, виконується другий цикл. Виконання набору операторів усередині блоку for триває до тих пір, поки *i* не досягне значення 100.

І в 99-й раз умовний вираз повертає true; програма виконується в останній раз. Значення *i* збільшується ще раз і стає рівним 100. Обчислення умовного виразу дає в результаті false, тому виконання циклу переривається. Управління переходить до операторам, наступним безпосередньо за закриває фігурної дужки.

Оскільки значення *i* було збільшено до 100 до завершення циклу, 100 і являє собою останнє значення *i*. Зверніть увагу, що у відповідність з правилами, розглянутими в розділі 5, і можна використовувати поза циклом.

#### ПРИМІТКА

Якщо умовний вираз повертає false при першому проході циклу, код в фігурних дужках виконуватися не БУДЕ.

Подібно операторам if, цикли for також можуть бути вкладеними. Лістинг 7.5 містить код проходження по кожній координаті сітки 10x10. Для кожної ітерації першого циклу мають місце 10 ітерацій вкладеного циклу. В результаті вкладений цикл буде виконуватися 100 раз.

#### Лістинг 2.6. Приклад вкладеного циклу.

```

<html>
<body>
  <script type="text/javascript">
    document.write("Все координати x, y між (0,0) і (9,9): <br>");
    for (var x = 0; x < 10; ++x) {
      for (var y = 0; y < 10; ++y) {
        document.write("(" + x + ", " + y + "), ");
      }
    }
  </script>

```



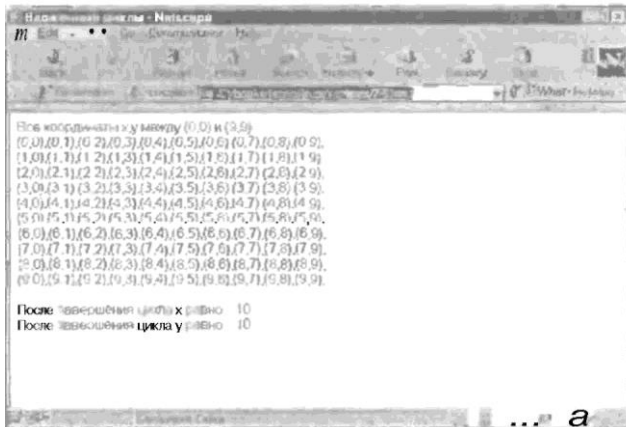
```

    }
    document.write('<br>');
}
document.write("<br> Після завершення циклу x одно:" + x);
document.write("<br> Після завершення циклу y одно;" + y);
</Script>
</Html>

```

Змінної *x* спочатку присвоюється значення 0. Потім JavaScript доходить до вкладеного циклу і привласнює змінній *y* також значення 0. Вкладений цикл відображає значення *x* і *y* і потім збільшує *y* на 1. Вкладений цикл триває до тих пір, поки *y* не стане рівним 10. В цей момент вже згенеровано 10 наборів координат. Виконання вкладеного циклу переривається і управління повертається до зовнішнього циклу. Значення *x* збільшується на 1, після чого знову запускається вкладений цикл.

JavaScript знає, що вкладений цикл необхідно почати з самого початку, в зв'язку з чим вираз ініціалізації має обчислюватися повторно. Таким чином, *y* присвоюється значення 0 і оператори виконуються ще 10 раз. Процес триває до тих пір, поки *x* не стане рівним 10 і виконання зовнішнього циклу, нарешті, завершується. В результаті відображається 100 наборів координат (див. Рис.2.7).



**МАЛЮНОК 2.7.** 100 пар координат, згенерованих в двох *циклах*.

Оскільки будь-яких обмежень на кількість вкладених циклів немає, отже, число координат можна збільшити до трьох або, скажімо, до чотирьох. Даний метод можна використовувати для звернення до елементів багатовимірного масиву.

### **For..in**

Для використання циклу *for..in* будуть потрібні знання об'єктів JavaScript.

Конструкція *for..in* надає можливість виконання набору операторів для кожного властивості об'єкта. Цикл *for..in* можна використовувати з будь-яким об'єктом JavaScript, незалежно від його властивостей. Якщо об'єкт не має властивостей, цикл не виконується. Цикл *for..in* працює також і з одними об'єктами. Мінлива призначеного для користувача об'єкта JavaScript розглядається як властивість, і тому для кожної з них виконується цикл. Синтаксис виглядає так:

#### **for (властивість in об'єкт)**

{оператори }

Тут властивість - це строковий літерал, згенерований JavaScript. При кожному проході циклу властивості присвоюється ім'я наступного властивості, що міститься в об'єкті, і так до тих пір, поки не переберуться всі властивості. У лістингу 2.7 така функціональність застосовується для відображення імен та значень всіх властивостей об'єкта *Document* (див. Рис. 2.8).



МАЛЮНОК 2.8. Список властивостей об'єкта Document разом з їх значеннями.

Лістинг 2.7. Використання циклу for..in.

```
<Html>
<body>
  <Script type="text/javascript">
    //Оголошення змінних
    var anObject = document;
    var propertyInfo = "";
    for (var propertyName in anObject) {
      propertyInfo = propertyName + " - " + anObject[propertyName];
      document.write(propertyInfo + "<br>");
    }
  </Script>
</Html>
```

## while

Оператор while діє подібно циклу for, проте не включає в себе функції ініціалізації інкременту змінних під час їх оголошення.

Змінні необхідно оголошувати заздалегідь, а інкремент або декремент їх визначати в рамках блоку операторів. Розглянемо синтаксис while:

```
while (Вираз_умови)
{оператори
}
```

Лістинг 2.8 надає приклад використання логічної змінної в якості флагоу, що визначає продовження виконання циклу. Ця змінна, status, оголошена до початку циклу і їй присвоєно значення true. Як тільки і стане дорівнює 10, status отримає значення false, після чого цикл завершується. Результат лістингу 2.8, як показано на рис. 2.9, - це сума цілих чисел від 0 до 10.

Лістинг 2.8. Використання циклу while. \_\_\_\_

```
<Html>
<body>
  <Script type="text/javascript">
    i = 0;
    var result = 0;
    var status = true;
    document.write("0");
    while (status) {
      result += ++i;
      document.write("+" + i);
      if (i = 10) {
        status = false;
      }
    }
  </Script>
</body>
```



```

    }
}
document.writeln("=" + result);
</Script>
</Html>

```

### Do...while

Цей цикл працює подібно циклу **while** за виключенням того, що умовний вираз перевіряється лише в кінці циклу, тобто після першої ітерації циклу. Цей спосіб гарантує, що набір операторів, що знаходяться в межах фігурних дужок, буде виконаний принаймні один раз.



**МАЛЮНОК 2.9.** Результат виконання лістингу 2.8 - одинадцять ітерацій циклу **while**.

Лістинг 2.8 містить приклад, в якому заздалегідь невідомо, яке значення може прийняти userEntry (як якщо б згадане значення вводилося користувачем). userEntry присвоюється значення 0 просто для демонстрації того, що користувач цілком може ввести значення, яке не буде задовольняти висловом умови.

**Лістинг 2.8.** Оператор **do..while**, що забезпечує принаймні один прохід циклу.

```

<Html>
<body>
  <Script type="text/javascript">
    userEntry = 0;
    var x = 1;
    do {
      document.writeln(x); x++;
    } while (x <= userEntry);
  </Script>
</Html>

```

## break і continue

Використовуючи цикли, варто звернути увагу на те, що за замовчуванням цикл не припиняється, а буде виконуватися до тих пір, поки певна умова не дорівнюватиме false. Однак, іноді потрібно вийти з циклу до того, як він дійде до закриваючої фігурної дужки. Подібне досягається за рахунок розміщення в блок операторів усередині циклу break або continue.

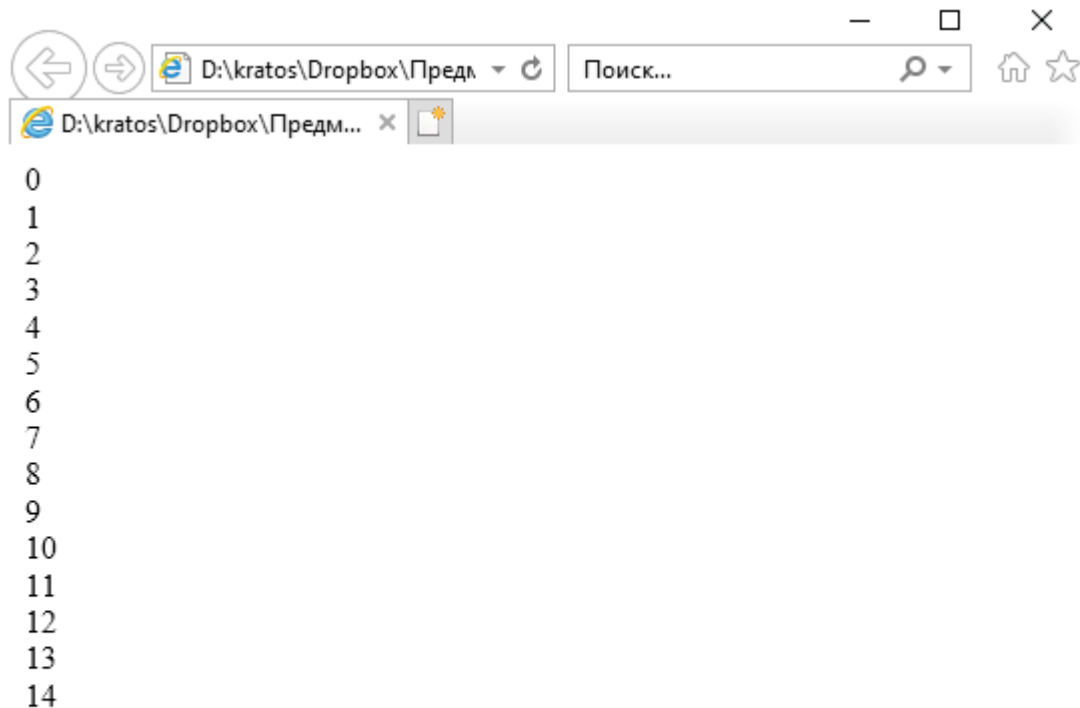
Оператор break завершує виконання циклу, в той час як continue пропускає інші оператори поточної ітерації, обчислює чергове значення виразу циклу (якщо таке існує) і починає виконання наступної ітерації. Відмінності між цими двома операторами продемонстровані в лістингу 2.9, який містить досить тривіальний сценарій обчислення наближеного цілочисельного значення квадратного кореня числа  $n$ .

### Лістинг 2.9. Використання операторів break і continue.

```
<Html>
<body>
  <Script type="text/javascript">
    // Оголошення змінних
    var highestNum = 0;
    var n = 175;    // тестове значення
    for (var i = 0; i < n; ++i) {
      document.write(i + "<br>");
      if (n < 0) {
        document.write("n не може бути негативним ");
        break;
      }
      if (i * i <= n) {
        highestNum = i;
        continue;
      }
      document.write("<br>Зроблено!");
      break;
    }
    document.write("<br>цілочисельне значення, не перевищує квадратний корінь
");
    document.write(" з " + n + "=" + highestNum);
  </Script>
</Html>
```

Спочатку  $i$  присвоюється 0, цикл починається і відображається значення  $i$ . Потім сценарій переконується, що  $n$  не є негативним числом. Якщо  $n$  негативне, цикл переривається і виконується залишок програми після закривання фігурної дужки. Якщо  $n$  позитивне  $i$  множиться на той же  $i$  та результат порівнюється з  $n$ . Якщо результат менше  $n$ ,  $i$  зберігається як найкраще поточне наближення квадратного кореня з  $n$ .

Після того оператор **continue** пропускає решту поточної ітерації і відновлює роботу циклу з моменту збільшення  $i$ . Як тільки квадрат  $i$  перевищить значення  $n$ , сценарій пропускає **continue** і переходить до оператора **break**, який зупиняє виконання циклу. Апроксимація квадратного кореня з 175 показана на рис 2.10.



Зроблено!

цілочисельне значення, не перевищує квадратний корінь з  $175=13$

Малюнок 2.10. Зовнішній вигляд вікна після того, як цикл переходить до оператора `break` та зупиняється.

### Мітки

Мітку можна поміщати перед будь-якою структурою, яка містить інші оператори. Це дозволяє перейти з рамок умовного оператора або циклу на зовсім визначене місце програми. Приклад застосування міток показаний в лістингу 2.10.

### Лістинг 2.10. Використання міток.

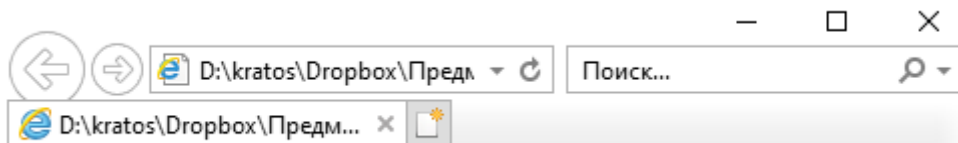
```
<html>
<body>
  <script type="text/javascript">
    // Оголошення змінних
    var stopX = 3;
    var stopY = 8;
    document.write("Усі пари x y між (0,0) і ( ");
    document.write(stopX + ", " + stopY + "): <br> ");
    loopX:
    for (var x = 0; x < 10; ++x) {
      for (var y = 0; y < 10; ++y) {
        document.write("(" + x + ", " + y + " ");
        if ((x = stopX) && (y = stopY)) {
          break loopX;
        }
      }
      document.write("<br>");
    }
    document.write("<br> Після завершення циклу x дорівнює: " + x);
```

```

        document.writeln("<br> Після завершення циклу у дорівнює:" + y);
    </Script>
</Html>

```

У лістингу 2.10 цикл **for** "позначений" призначеним для користувача ідентифікатором **loopX**. Це дозволяє залишати цикл **for** або, навпаки, продовжувати його незалежно від вкладеності. Та ж мітка **loopX** вказується в операторі **break**, щоб завершити обидва циклів **for**. Без цієї мітки оператор **break** зупинив би тільки цикл, що генерує значення для **y**. На рис. 2.11 показаний результат роботи програми, коли **x** і **y** досягають, відповідно, значень 3 і 8.



Усі пари **x** у між (0,0) і ( 3, 8):  
 (0,0) Після завершення циклу **x** дорівнює:3  
 Після завершення циклу **y** дорівнює:8

**МАЛЮНОК 2.11.** Використання оператора **label** для виходу з вкладеного циклу **for**.

ОпераTop with

Оператор **with** застосовується щоб уникнути багаторазового повторення посилання на об'єкт при доступі до його властивостей і методів. Будь-які властивості або методи в блоці **with**, які JavaScript не може впізнати, асоціюються з об'єктом, зазначеним в цьому блоці. Синтаксис **with** такий:

```

with (об'єкт) {оператори}

```

*об'єкт* визначає, який об'єкт необхідно використовувати в разі відсутності об'єктної посилання в блоці операторів. Даний оператор особливо корисний при використанні складних математичних функцій, які доступні тільки через об'єкт **Math**. Оскільки об'єкт **Math** буде розглядатися, починаючи з глави 10, тут показаний оператор **with** для вже знайомого об'єкта **document**.

При виклику методів **write()** або **writeln()**, що належать об'єкту **document**, перед ними необхідно вказувати префікс **document**:

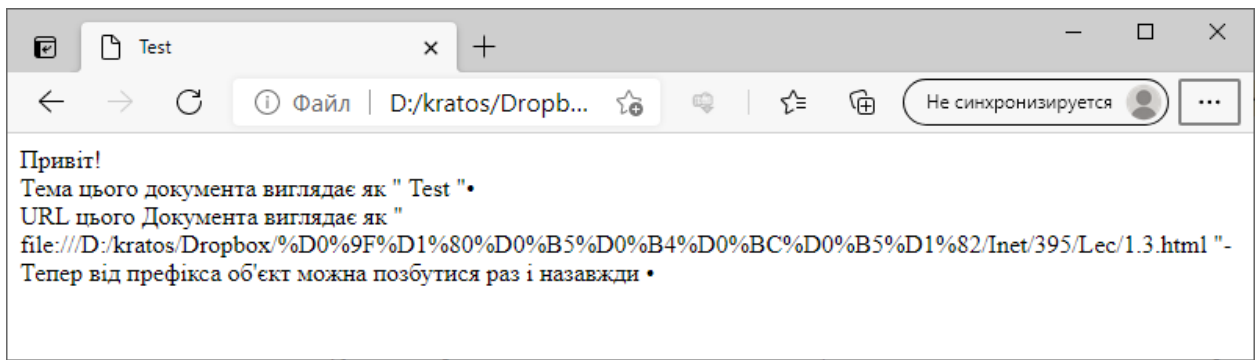
```

document .writeln ( "Привіт всім! ");

```

Подібна методика застосовується досить часто і у великих кількостях. Для економії на обсязі коду всі оператори, які посилаються на об'єкт **document**, варто помістити в блок **with**. Як це зробити показано в лістингу 7.11. Роблячи таким чином, можна позбутися від настирливого префікса **document** в посиланнях на методи і властивості документа.

Зверніть увагу, що **title** і **URL** - це властивості об'єкта **document**, які повинні записуватися як **document.title** і **document.URL**. При використанні оператора **with** необхідність в префіксі виникає тільки один раз, проте, досягається той самий результат, що і при наборі префіксу в кожному рядку. На рис. 2.11 показаний результат виконання коду з лістингу 2.12



МАЛЮНОК 2.12. Відображення інформації з використанням оператора with.

### Лістинг 2.11. Використання оператора with в JavaScript.

```
<html>
<head>
  <meta charset="windows1251" />
  <title>Test</title>
</head>
<body>
  <Script type="text/javascript">
    with (document) {
      write("Привіт!");
      write("<br> Тема цього документа виглядає як \" " + title + " \".");
      write("<br> URL цього Документа виглядає як \" " + URL + " \".");
      write("<br>Тепер від префікса об'єкт можна позбутися раз і назавжди •
    ");
    }
  </Script>
</body>
</Html>
```

#### ПРИМІТКА

Залежно від конкретного браузера, приклад може відобразити URL в закодованому форматі.

#### Оператор switch

Оператор switch використовується для порівняння одного значення з великою кількістю інших.

У лістингу 2.13 передбачається, що змінна request може змінюватися в залежності від вимог користувача. В даному прикладі request присвоюється тестове значення "Name".

### Лістинг 2.13. Оператор switch.

```
<html>

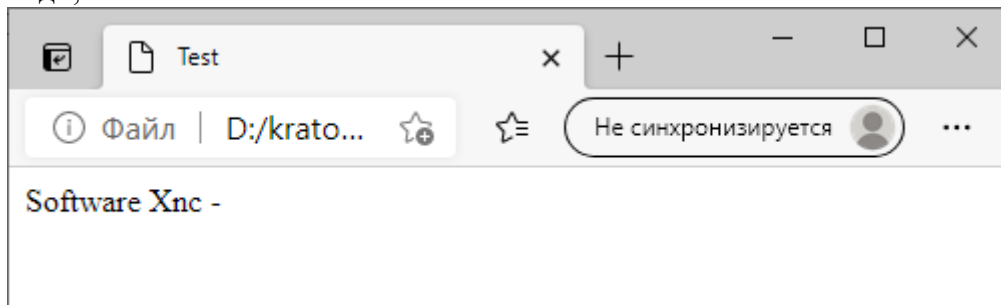
<head>
  <meta charset="windows1251" />
  <title>Test</title>
</head>
<body>
  <Script type="text/javascript">
    var request = "Name";
    switch (request) {
      case "Logo":
        document.write('<img src = "logo.gif" alt = "Logo"> ');
        document.write("<br>");
        break;
```

```

        case "Name":
            document.write("Software Xnc -");
            document.write("<br>");
            break;
        case "Products":
            document.write("MyEditor");
            document.write("<br>");
            break;
        default:
            document.write("www.mysite.com");
            break;
    }
</Script>
</body>
</Html>

```

Друга конструкція case збігається з поточним значенням request, отже, виконуватися команди, з неї.



МАЛЮНОК 2.12. Використання оператора switch.

Оператор break використовується для переривання подальшого виконання коду, що залишився в switch. Якби оператор break не використовувався залишився код виконувався б для кожного випадку, незалежно від відповідності між request і case. З рис. 2.12 ясно, що request відповідає значенню "Name", при цьому на екран виводиться назва компанії.

#### Контрольні питання.

1. **Арифметичні оператори:** Які основні арифметичні оператори (+, -, \*, /, %) доступні в JavaScript? Як працює оператор декременту (--), та інкременту (++)?
2. **Оператори порівняння:** Поясніть дію операторів == (рівність), != (нерівність), === (строга рівність), > (більше), < (менше). Що повертають ці оператори?
3. **Логічні оператори:** Як працюють логічне "І" (&&), логічне "АБО" (||) та заперечення (!)? Наведіть приклади їх використання в умовах.
4. **Умовний оператор if...else:** Опишіть синтаксис конструкції if...else. Як додати додаткові умови за допомогою else if?
5. **Тернарний оператор:** Як працює умовний оператор ? : (наприклад, condition ? val1 : val2)?
6. **Оператор switch:** Для чого використовується конструкція switch? Яку роль відіграють ключові слова case, break та default?
7. **Цикл for:** Опишіть структуру циклу for (ініціалізація, умова, ітерація). У яких випадках цей цикл є найбільш доречним?
8. **Цикл while:** Як працює цикл while? Чим він відрізняється від циклу do...while (особливо щодо виконання тіла циклу хоча б один раз)?
9. **Цикл for...in:** Для чого використовується цикл for (var prop in object)? Що саме перебирає змінна в цьому циклі?

10. **Оператори break та continue:** У чому різниця між break (повний вихід з циклу) та continue (перехід до наступної ітерації)?
11. **Мітки (Labels):** Як використання міток разом з break дозволяє вийти з вкладеного циклу відразу на кілька рівнів вгору?
12. **Оператор typeof:** Що повертає оператор typeof для різних типів даних (числа, рядка, об'єкта, невизначеної змінної)?
13. **Оператор with:** Для чого призначений оператор with (наприклад, with(document)) і як він спрощує звернення до властивостей об'єкта?